Flexbox

Complete Visual + Practical Mastery Guide

Everything from your original deck, plus the parts every real project needs: flex-basis, the flex shorthand, order, gap, common gotchas, and 4 practical layouts you can build today.

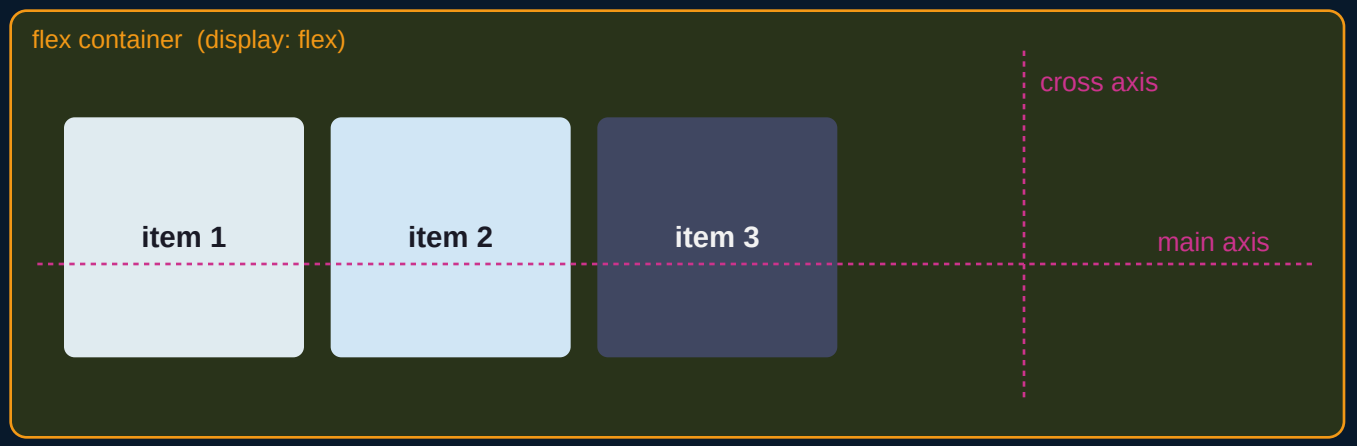
This guide uses the same visual language throughout:

01 · Foundations

What is Flexbox

A one-dimensional layout system

Flexbox arranges items along a single line, either a row or a column. The browser decides how much space each item gets based on the rules you set on the container and, optionally, on individual items.



Two axes to know:

Main axis - the direction items line up (set by flex-direction).

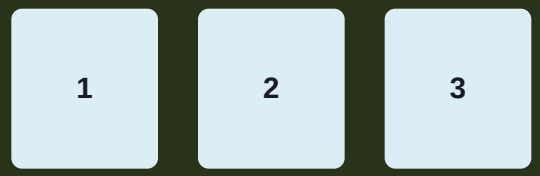
Cross axis - perpendicular to the main axis (used by align-items).

02 · Container properties

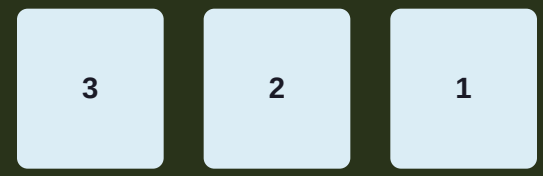
flex-direction

Sets the main axis and the direction items flow

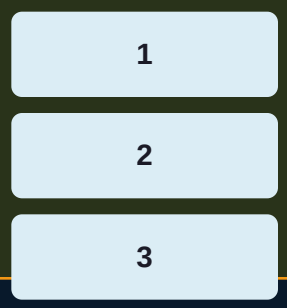
row (default)



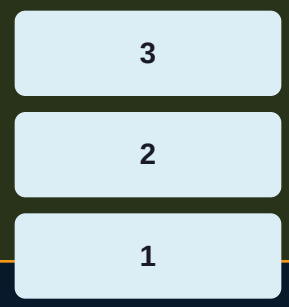
row-reverse



column



column-reverse



02 · Container properties

justify-content

Aligns items along the MAIN axis

flex-start

flex-end

center

space-between

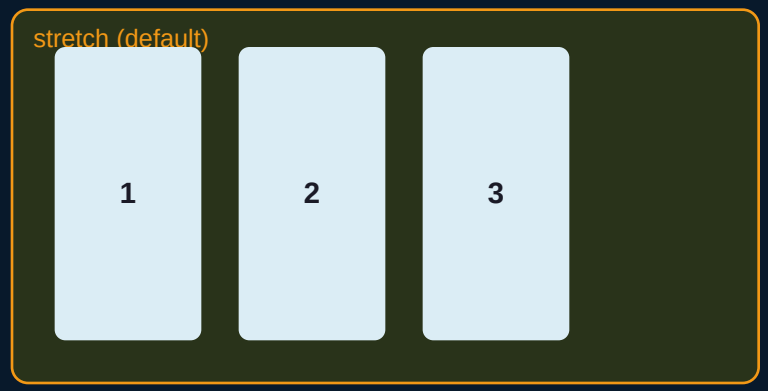
space-around

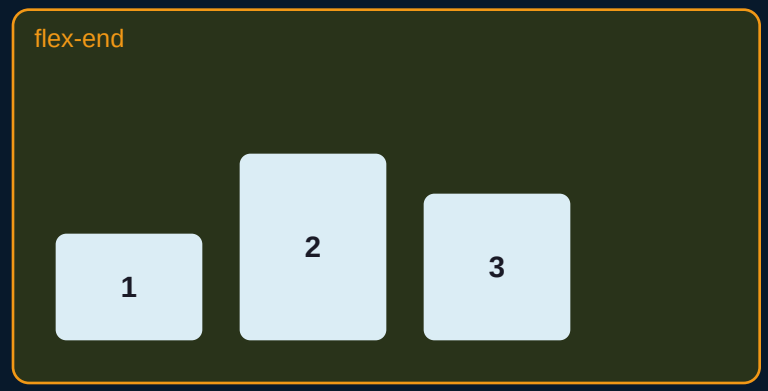
space-evenly

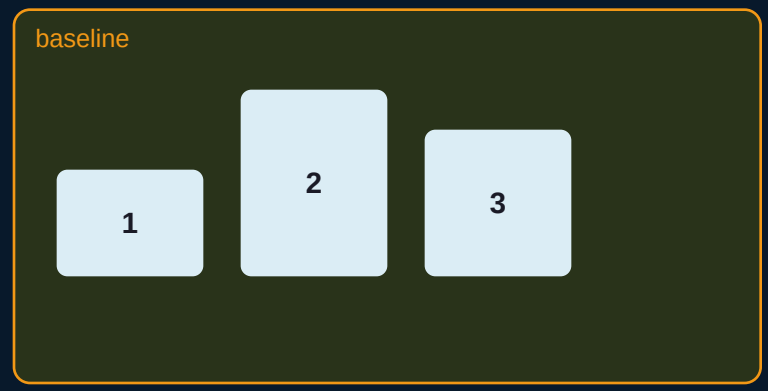
02 · Container properties

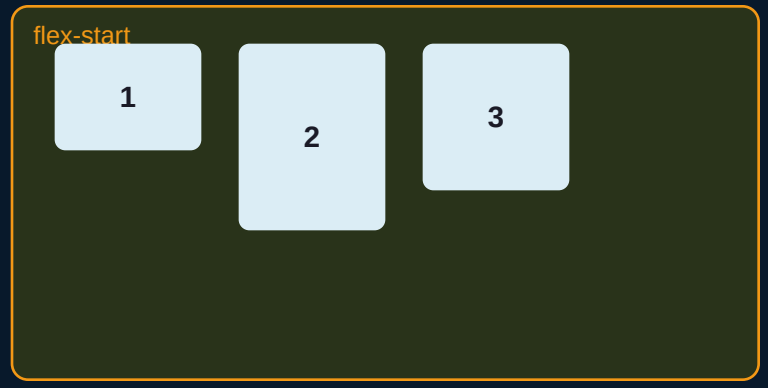
align-items

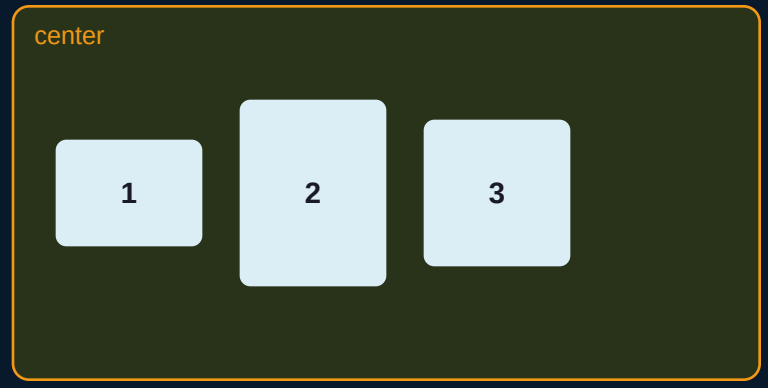
Aligns items along the CROSS axis (single line)









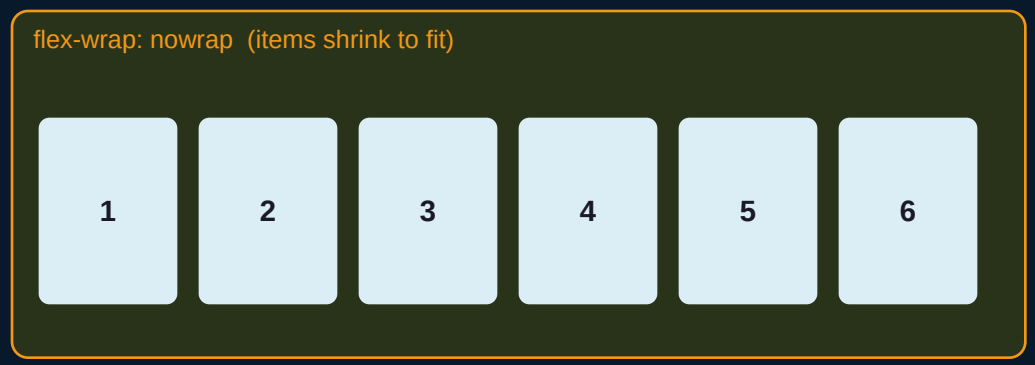


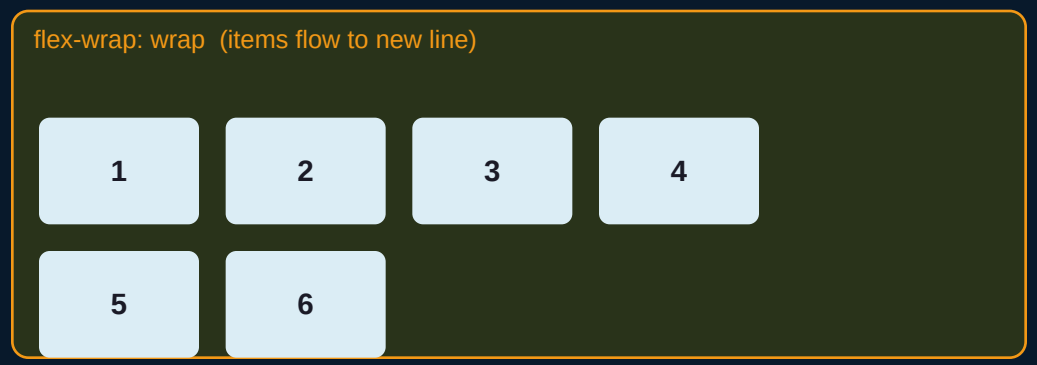
02 · Container properties

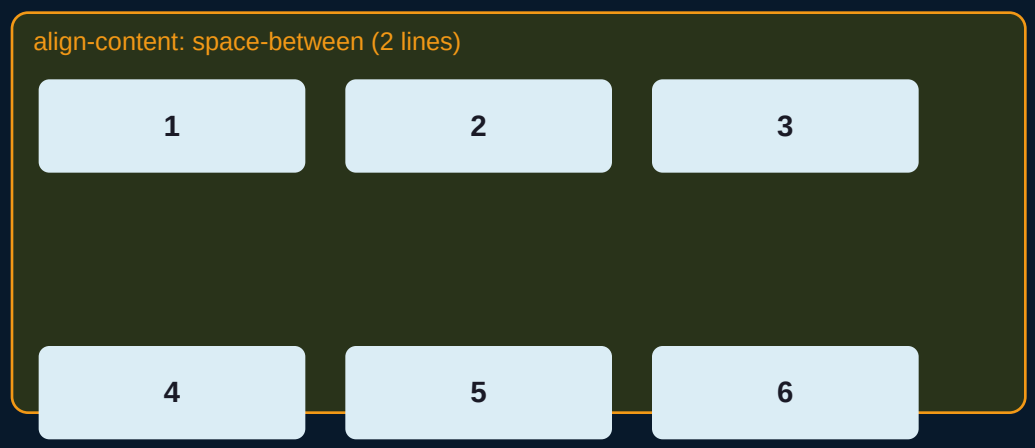
flex-wrap & align-content

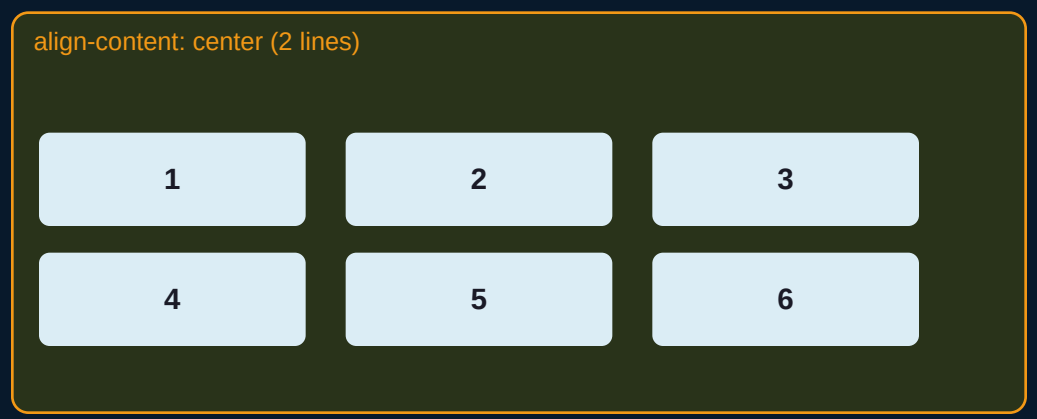
Control what happens when items don't fit on one line

flex-wrap decides IF items wrap to new lines. align-content then positions those wrapped lines along the cross axis (it only has an effect when there are 2+ lines).







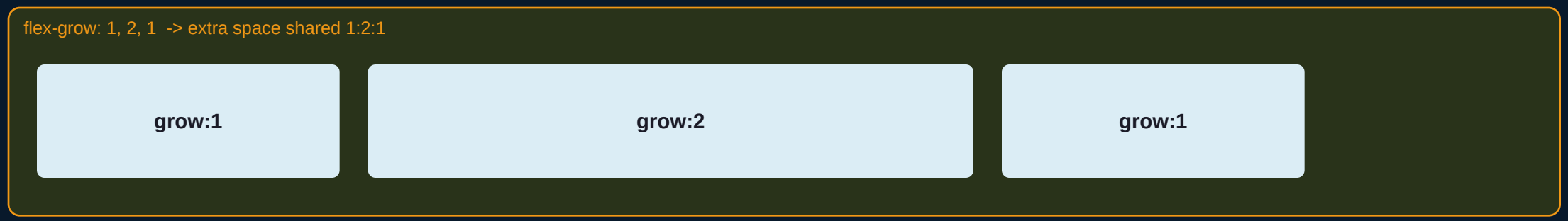


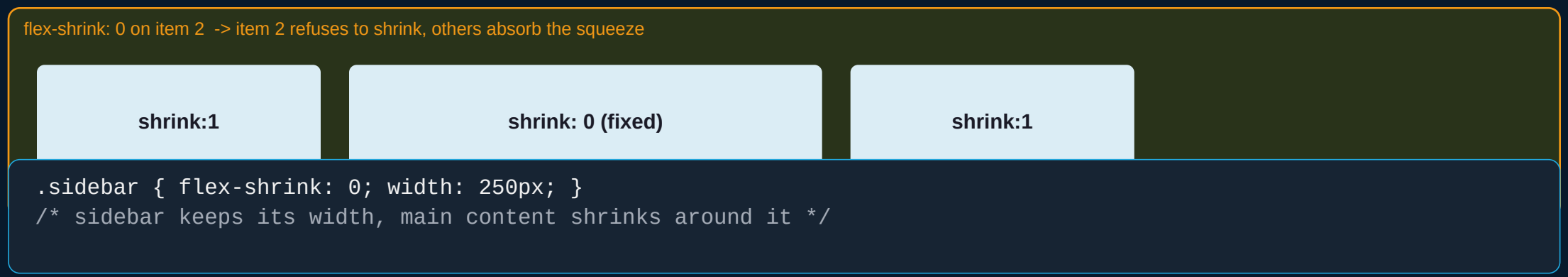
03 · Item properties

flex-grow & flex-shrink

How items grow or shrink to fill / fit the container

flex-grow distributes EXTRA space. flex-shrink removes space PROPORTIONALLY when items overflow. Both default to a ratio - an item with flex-grow: 2 takes twice the extra space of one with flex-grow: 1.






```
.sidebar { flex-shrink: 0; width: 250px; }
/* sidebar keeps its width, main content shrinks around it */
```

03 · Item properties · NEW

flex-basis & the flex shorthand

Missing from most tutorials, and used in almost every real project

flex-basis sets an item's STARTING size before grow/shrink are applied - like width, but on the main axis. Instead of writing all three properties separately, use the flex shorthand.

```
flex: <grow> <shrink> <basis>;
/* long form */
flex-grow: 1;
flex-shrink: 1;
flex-basis: 200px;

/* shorthand - same result */
flex: 1 1 200px;
```

Common shorthand keywords

```
flex: 1;        /* = 1 1 0%   -> grow, ignore basis    */
flex: auto;     /* = 1 1 auto -> grow, basis = content */
flex: none;     /* = 0 0 auto -> rigid, never resizes  */
flex: initial;  /* = 0 1 auto -> default browser value */
```

Rule of thumb:

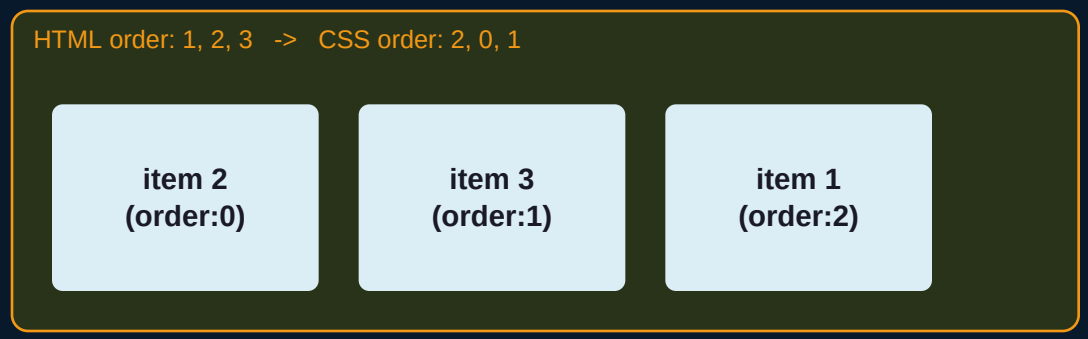
Use flex: 1 for items that should share space equally. Use flex: 0 0 auto (or flex-shrink: 0) for things like icons or fixed sidebars that must never resize.

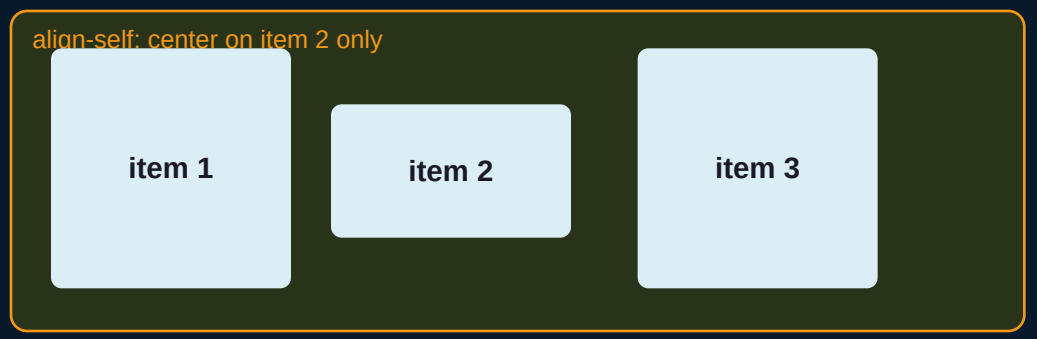
03 · Item properties · order is NEW

order & align-self

Per-item overrides, without touching your HTML

order changes the VISUAL order of an item without reordering the HTML - useful for moving something first on mobile and last on desktop. align-self overrides align-items for one single item.





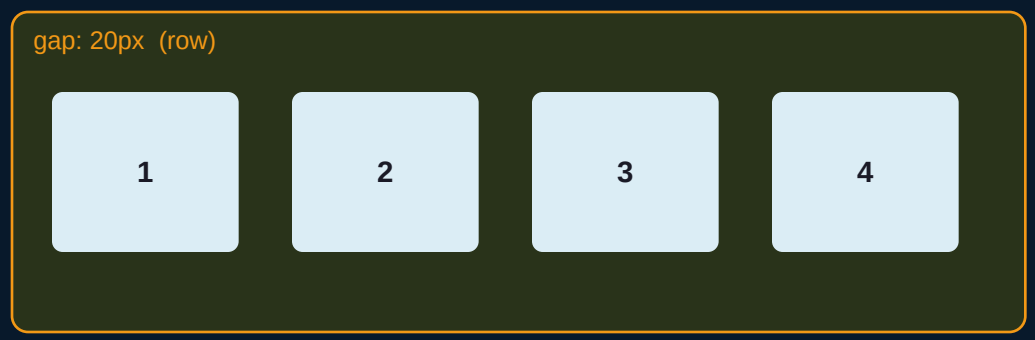

```
.item-2 { order: -1; }   /* moves item to the front, negative goes first */
.item-2 { align-self: flex-end; }  /* only this item ignores align-items */
```

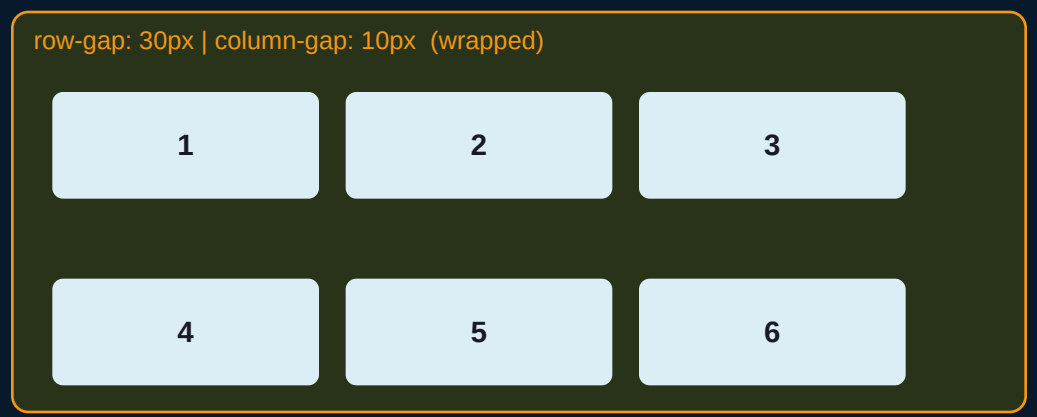
04 · Spacing · NEW

gap, row-gap, column-gap

The modern replacement for margin hacks

gap adds space BETWEEN flex items only - never on the outer edge of the container. This means you no longer need :not(:last-child) margin tricks to space out a row of items.





Before vs after

```
/* old way - hacky */
.item { margin-right: 16px; }
.item:last-child { margin: 0; }

/* new way - clean */
.container { display: flex; gap: 16px; }
```

Practical pattern 1 - perfect centering

The most common flexbox use case in the real world

Centered box, any screen size

I'm centered

```css
.center-wrapper {
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
}
```

This 4-line pattern replaces old centering hacks (absolute positioning + negative margins, or transform: translate). It works regardless of the child's width or height.

Practical pattern 2 - navigation bar

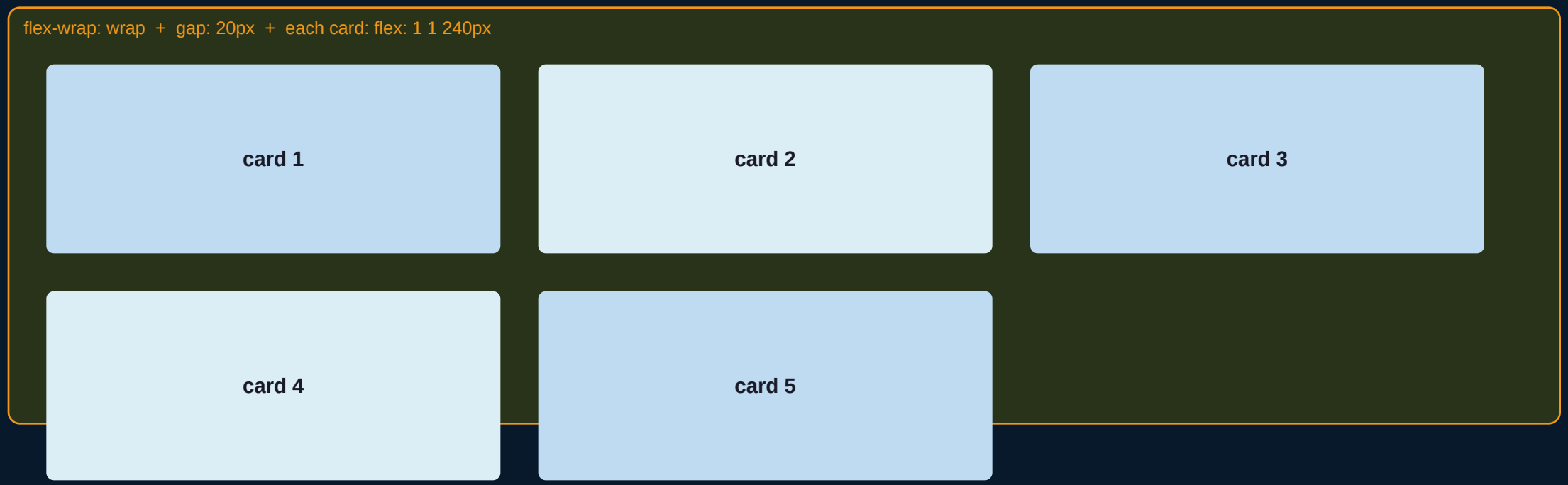
Logo on the left, links on the right, one line of CSS

CSS

```
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 16px 24px;
}
.nav-links { display: flex; gap: 20px; } /* nested flex for the links group */
```

Practical pattern 3 - responsive card grid

Cards that wrap and grow to fill the row evenly

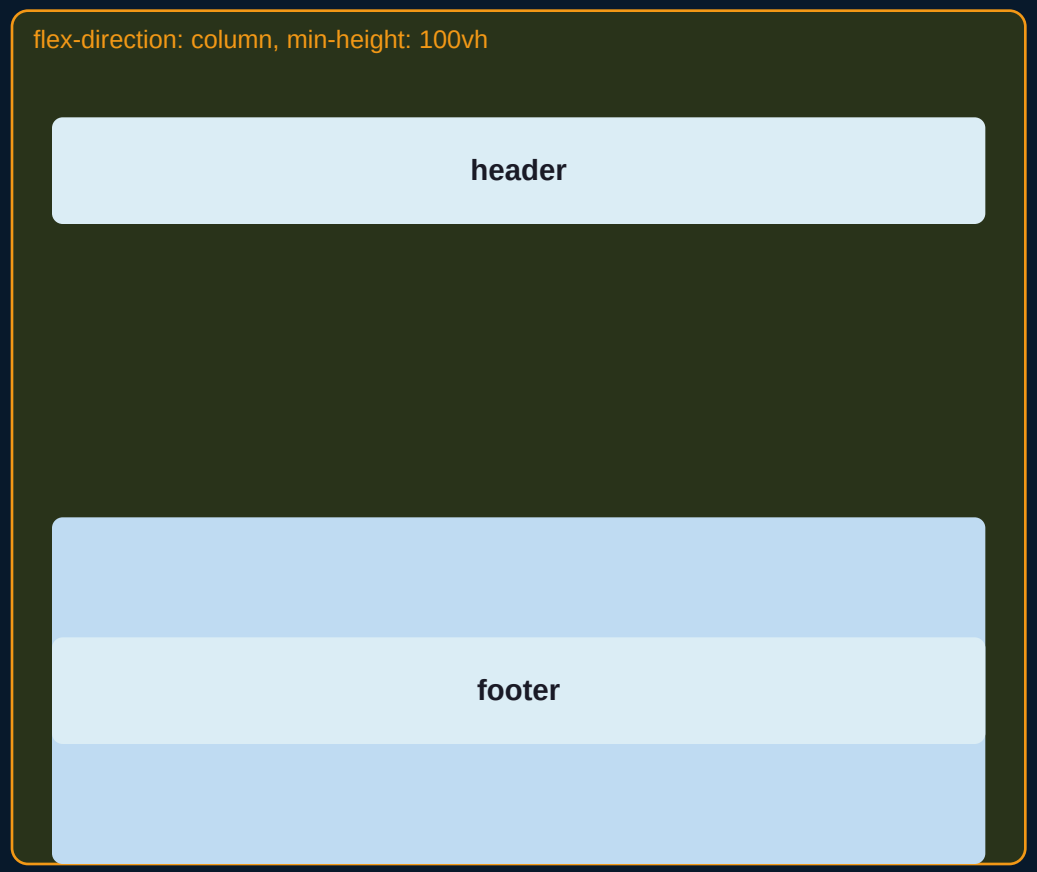


CSS

```
.card-grid { display: flex; flex-wrap: wrap; gap: 20px; }
.card { flex: 1 1 240px; }  /* grow, shrink, but never smaller than 240px - wraps automatically */
```

Practical pattern 4 - sticky footer layout

Footer stays at the bottom even on short pages




```css
.page {
  display: flex;
  flex-direction: column;
  min-height: 100vh;
}
.main { flex: 1; }  /* takes all remaining space, pushing footer
```

06 · Debugging · NEW

Common gotchas

The mistakes that trip up almost everyone at first

Text/images won't shrink below their content size

Flex items have an implicit min-width: auto / min-height: auto. Add min-width: 0 (or min-height: 0) to let them shrink properly - common fix for overflowing text or images in a flex row.

justify-content does nothing

It only works if there's leftover space on the main axis. If items already fill the container (or use flex-grow), there's nothing left to distribute.

align-items: baseline looks wrong

Baseline aligns the TEXT baselines, not the box edges - expected behavior when items have different font sizes or padding.

Wrapped items don't line up in a grid

Flexbox is one-dimensional - each line sizes independently. For a true 2D grid where columns line up, use CSS Grid instead.

07 · Reference

Cheat sheet

Every property in this guide, at a glance

CONTAINER PROPERTIES

`display: flex;`
Turn on flexbox

`flex-direction`
row / row-reverse / column / column-reverse

`flex-wrap`
nowrap / wrap / wrap-reverse

`justify-content`
align on the main axis

`align-items`
align on the cross axis (one line)

`align-content`
align on the cross axis (multiple lines)

`gap / row-gap / column-gap`
space between items

ITEM PROPERTIES

`flex-grow`
share of extra space

`flex-shrink`
share of space removed on overflow

`flex-basis`
starting size before grow/shrink

`flex: g s b;`
shorthand for the three above

`order`
visual reorder without touching HTML

`align-self`
override align-items for one item